

Spis treści

Projekt układu sumy liczby 2-bitowej z 3-bitową na bramkach

1	Wstęp teoretyczny	2
1.1	Definicja sumatora cyfrowego	2
1.2	Zasada działania sumatora liczby 2-bitowej z liczbą 3-bitową	2
1.3	Zastosowanie sumatorów cyfrowych	3
1.4	Rodzaje sumatorów	3
2	Opis techniczny projektu	4
2.1	Założenia projektowe	4
2.2	Specyfikacja wejść i wyjść	4
2.3	Analiza logiczna układu	6
2.4	Prezentacja układu i przejście do dalszej części pracy	6
3	Minimalizacja funkcji logicznej	7
3.1	Funkcje wyjściowe układu	7
3.2	Siatka Karnaugh dla funkcji S_0	8
3.3	Siatka Karnaugh dla funkcji S_1	8
3.4	Siatka Karnaugh dla funkcji S_2	8
3.5	Siatka Karnaugh dla funkcji S_3 oraz przeniesienia P_1	9
3.6	Postać minimalna i realizacja na bramkach NOR	10
3.7	Porównanie liczby bramek przed i po minimalizacji	11
4	Schematy logiczne, tabelaryczne i wykresowe	11
4.1	Schemat logiczny układu	11
4.2	Tablica prawdy	12
4.3	Schemat blokowy układu	14
4.4	Wykres czasowy i opis działania	14
5	Symulacja i testowanie	15
5.1	Środowisko symulacyjne	15
5.2	Przygotowanie układu do symulacji	16
5.3	Scenariusze testowe	16
5.4	Oczekiwane wyniki symulacji	16
5.5	Analiza wyników	16
6	Podsumowanie i wnioski	18
6.1	Ocena zaprojektowanego układu	18
6.2	Korzyści wynikające z minimalizacji bramek NOR	18
6.3	Ograniczenia rozwiązania	18
6.4	Możliwości rozwoju projektu	19
6.5	Zakres pracy	19
7	Źródła	19

1 Wstęp teoretyczny

1.1 Definicja sumatora cyfrowego

Sumator cyfrowy jest układem kombinacyjnym służącym do wykonywania dodawania liczb zapisanych w postaci binarnej. Jego zadaniem jest wyznaczenie wyniku sumy oraz ewentualnego przeniesienia na podstawie wejściowych słów bitowych. W zależności od zaprojektowanej architektury sumator może realizować dodawanie dwóch liczb bez przeniesienia, z jednym przeniesieniem właściwym lub z mechanizmem przeniesienia wyprowadzonym na wyjście.

W niniejszym projekcie analizowany jest układ dodający liczbę dwubitową z liczbą trójbitową, dla których wynik sumy może przyjąć wartość 4-bitową. Maksymalna suma wynosi $3 + 7 = 10 = 1010_2$, więc wynik wymaga czterech bitów. Takie zadanie wymaga uwzględnienia możliwości wystąpienia przeniesienia z najstarszego bitu wyniku, co w naturalny sposób prowadzi do konieczności wyznaczenia czterech bitów wyniku: trzech bitów sumy właściwej oraz jednego bitu przeniesienia.

Dla przyjętego rozwiązania:

- liczba pierwsza (dwubitowa) przedstawiana jest bitami A_1, A_0 (gdzie A_1 jest bitem starszym, a A_0 bitem młodszy),
- liczba druga (trójbitowa) przedstawiana jest bitami B_2, B_1, B_0 ,
- wynik sumy o wartości 4-bitowej oznaczany jest jako $S_3S_2S_1S_0$,
- bity S_2, S_1, S_0 odpowiadają za wynik dodawania, a bit S_3 odpowiada za najstarsze przeniesienie.

Wyjście układu oznaczone jako S_3, S_2, S_1, S_0 przyjmuje wartości zależne od sumy wejściowych liczb. Najstarszy bit wyniku S_3 przyjmuje wartość logiczną 1 wtedy, gdy suma liczb wejściowych jest większa lub równa 8; w pozostałych przypadkach przyjmuje wartość 0.

Suma liczb dwubitowej i trójbitowej może być przedstawiona jako:

$$A + B = S \quad (1)$$

gdzie:

- $A = 2A_1 + A_0$,
- $B = 4B_2 + 2B_1 + B_0$,
- $S = 8S_3 + 4S_2 + 2S_1 + S_0$.

Sumator może być zrealizowany przy wykorzystaniu podstawowych bramek logicznych, takich jak AND, OR oraz NOT, a także bramek XOR. W projekcie szczególną uwagę poświęcono minimalizacji liczby wykorzystanych bramek logicznych, przy jednoczesnym ograniczeniu do realizacji wyłącznie na bramkach NOR, co wprowadza dodatkowe wymagania w zakresie przekształcania funkcji logicznych do postaci przyjaznej dla tego typu bramek.

1.2 Zasada działania sumatora liczby 2-bitowej z liczbą 3-bitową

Dodawanie liczb binarnych o różnych długościach polega na sumowaniu odpowiadających sobie bitów z uwzględnieniem przeniesienia z niższego poziomu na wyższy. W najprostszej formie można to realizować przy użyciu podstawowych bramek logicznych, w tym bramek XOR, AND oraz OR. Jednak w niniejszym projekcie przyjmuje się ograniczenie do realizacji wyłącznie na bramkach NOR, co wymaga odpowiedniego przekształcenia i zminimalizowania funkcji logicznych opisujących układ.

Dla liczb:

$$A = (A_1 A_0), \quad B = (B_2 B_1 B_0)$$

należy wykonać dodawanie pozycyjne, począwszy od bitu najmłodszej pozycji, a następnie przenieść ewentualne przeniesienie na bit starszy. W rezultacie otrzymujemy:

- S_0 – wynik sumy bitów najmłodszych A_0 i B_0 z uwzględnieniem przeniesienia 0 na tym poziomie,
- S_1 – wynik sumy bitów A_1 , B_1 oraz przeniesienia P_0 z niższego poziomu,
- S_2 – wynik sumy bitu B_2 oraz przeniesienia P_1 z niższego poziomu,
- S_3 – przeniesienie z najstarszego poziomu.

W praktyce realizacji na bramkach NOR każda z tych funkcji może być przekształcana do postaci sumy iloczynów lub iloczynu sum, a następnie minimalizowana w taki sposób, aby struktura miała możliwość realizacji przy użyciu wyłącznie bramek NOR.

Można to opisać za pomocą schematu działania układu w następujący sposób:

1. poziom młodszy: sumowanie A_0, B_0 z uzyskaniem S_0 oraz przeniesienia P_0 ,
2. poziom środkowy: sumowanie A_1, B_1 oraz przeniesienia P_0 z uzyskaniem S_1 i przeniesienia P_1 ,
3. poziom starszy: sumowanie B_2 oraz przeniesienia P_1 z uzyskaniem S_2 i przeniesienia S_3 .

1.3 Zastosowanie sumatorów cyfrowych

Sumatory cyfrowe są jednym z podstawowych elementów układów cyfrowych i mikroelektronicznych. Ich zastosowanie obejmuje:

- procesory i jednostki arytmetyczno-logiczne,
- układy sterowania i automatyki cyfrowej,
- systemy pomiarowe i cyfrowe przetworniki,
- tablice sumacyjne i magistrale danych,
- układy realizujące operacje arytmetyczne w strukturach programowalnych.

Układ realizujący dodawanie liczby dwubitowej z trójbitową z wynikiem 4-bitowym może być wykorzystany jako element podstawowy w szerszych strukturach licznikowych, sumacyjnych czy porównawczych. W projektach opartych na bramkach NOR istotne jest zachowanie minimalizacji, dzięki czemu układ staje się mniej skomplikowany pod względem liczby elementów, a jednocześnie łatwiejszy w interpretacji i dalszym skalowaniu.

1.4 Rodzaje sumatorów

Ze względu na sposób generowania przeniesienia można wyróżnić:

- sumator poziomy (seriowy), w którym przeniesienie jest propagowane kolejno między poziomami,
- sumator równoległy, w którym wszystkie bity są dostępne jednocześnie,

- sumator z generowaniem przeniesienia (carry look-ahead) – w przypadku większej liczby bitów zwiększa szybkość działania.

W niniejszym projekcie przyjęto układ równoległy, w którym liczba dwubitowa jest dodawana do liczby trójbitowej, z wyjściem 4-bitowym, realizowany na bramkach NOR. Układ charakteryzuje się prostą strukturą trójpoziomową, którą można łatwo zminimalizować pod kątem liczby bramek oraz liczby połączeń.

W kolejnym rozdziale przedstawiono opis techniczny projektu: przyjęte założenia projektowe, specyfikację wejść i wyjść oraz analizę logiczną układu, stanowiącą punkt wyjścia do minimalizacji funkcji logicznych.

2 Opis techniczny projektu

2.1 Założenia projektowe

Celem projektu jest zaprojektowanie układu kombinacyjnego realizującego funkcję sumy liczby dwubitowej z liczbą trójbitową, a następnie zminimalizowanie liczby wykorzystanych bramek logicznych przy zachowaniu realizacji wyłącznie na bramkach NOR.

Projektowany układ będzie dodawał dwie liczby binarne o różnych długościach:

$$A = (A_1 A_0), \quad B = (B_2 B_1 B_0)$$

Układ posiada pięć wejść binarnych oraz wyjście 4-bitowe. Wynik będzie informował o sumie liczb wraz z ewentualnym przeniesieniem najstarszego bitu.

Przyjęto następujące założenia:

- liczba A ma długość 2 bitów, a liczba B – 3 bity,
- wszystkie wejścia przyjmują wartości logiczne 0 lub 1,
- układ jest układem kombinacyjnym,
- wynik sumy ma 4 bity: $S_3 S_2 S_1 S_0$,
- stan wyjścia S_3 oznacza przeniesienie z najstarszego poziomu,
- maksymalna wartość sumy wynosi $3 + 7 = 10 = 1010_2$,
- podstawowym celem projektu jest minimalizacja liczby bramek logicznych,
- poprawność działania zostanie sprawdzona za pomocą tablicy prawdy i symulacji.

Minimalizacja układu ma na celu zmniejszenie jego złożoności. Mniejsza liczba zastosowanych bramek oznacza prostszy schemat logiczny, mniejsze zapotrzebowanie na elementy oraz potencjalnie mniejsze opóźnienia propagacji sygnału.

2.2 Specyfikacja wejść i wyjść

Projektowany sumator posiada pięć wejść:

- A_1 – starszy bit liczby A ,
- A_0 – młodszy bit liczby A ,

- B_2 – najstarszy bit liczby B ,
- B_1 – bit środkowy liczby B ,
- B_0 – najmłodszy bit liczby B .

Wyjścia układu to:

- S_0 – bit wyniku najmłodszy,
- S_1 – bit wyniku pośredni,
- S_2 – bit wyniku starszy,
- S_3 – najstarszy bit wyniku (przeniesienie).

Zależność pomiędzy wejściami a wyjściami można zapisać jako:

$$(A_1A_0)_2 + (B_2B_1B_0)_2 = (S_3S_2S_1S_0)_2 \quad (2)$$

Przykładowo, dla:

$$A = 10_2, B = 011_2$$

otrzymujemy:

$$S = 1001_2$$

Natomiast dla maksymalnych wartości:

$$A = 11_2, B = 111_2$$

otrzymujemy:

$$S = 1010_2$$

Tabela sygnałowa przedstawia podstawowe sygnały wejściowe i wyjściowe projektowanego układu:

Tabela 1: Sygnały wejściowe i wyjściowe sumatora liczby 2-bitowej z 3-bitową

Sygnał	Rodzaj	Opis
A_1	wejście	starszy bit liczby A (2-bitowej)
A_0	wejście	młodszy bit liczby A (2-bitowej)
B_2	wejście	najstarszy bit liczby B (3-bitowej)
B_1	wejście	bit środkowy liczby B (3-bitowej)
B_0	wejście	najmłodszy bit liczby B (3-bitowej)
S_3	wyjście	najstarszy bit wyniku (przeniesienie)
S_2	wyjście	bit wyniku starszy
S_1	wyjście	bit wyniku pośredni
S_0	wyjście	bit wyniku najmłodszy

2.3 Analiza logiczna układu

Aby wyznaczyć wynik sumy, należy wykonać dodawanie pozycyjne, począwszy od poziomu najmłodszego bitu.

Dla bitów najmłodszych:

$$S_0 = A_0 \oplus B_0 \quad (3)$$

przy czym w układzie opartym na bramkach NOR będzie to realizowane poprzez przekształcenie funkcji XOR na postać przyjazną dla bramek NOR.

Generowanie przeniesienia na najmłodszym poziomie wynosi:

$$P_0 = A_0 \cdot B_0 \quad (4)$$

Dla poziomu środkowego:

$$S_1 = A_1 \oplus B_1 \oplus P_0 \quad (5)$$

oraz przeniesienie:

$$P_1 = (A_1 \cdot B_1) \vee (P_0 \cdot (A_1 \vee B_1)) \quad (6)$$

Dla poziomu najstarszego, na którym pozostaje tylko bit B_2 liczby B oraz przeniesienie P_1 :

$$S_2 = B_2 \oplus P_1 \quad (7)$$

$$S_3 = B_2 \cdot P_1 \quad (8)$$

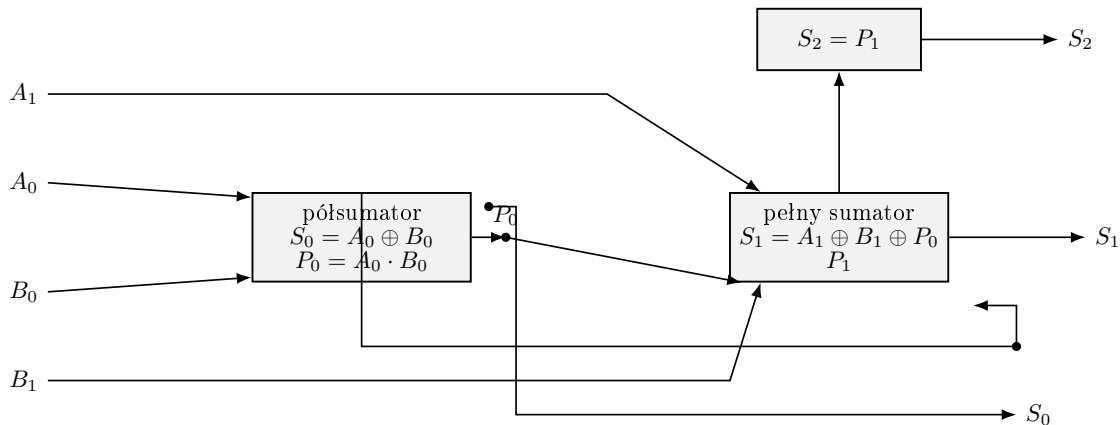
Funkcje te można zapisać w postaci sumy iloczynów lub iloczynu sum, a następnie zminimalizować do postaci przyjaznej dla bramek NOR.

Schemat działania można opisać następująco:

1. poziom młodszy: określenie S_0 oraz P_0 ,
2. poziom środkowy: określenie S_1 oraz P_1 ,
3. poziom starszy: określenie S_2 oraz S_3 .

2.4 Prezentacja układu i przejście do dalszej części pracy

Struktura układu przedstawiona jest na schemacie blokowym (rysunek 1), gdzie wydzielono trzy poziomy sumacyjne oraz wyjście przeniesienia.



Rysunek 1: Schemat blokowy sumatora liczby 2-bitowej z 3-bitową

Rysunek 1 przedstawia układ w formie hierarchicznej: wejścia A_0, B_0 są przetwarzane na poziomie młodszym, gdzie powstają S_0 oraz przeniesienie P_0 ; następnie bity A_1, B_1 wraz z P_0 wchodzi na poziom środkowy, gdzie powstają S_1 oraz P_1 ; na końcu bit B_2 wraz z P_1 wchodzi na poziom starszy, dając S_2 oraz S_3 . Wynik końcowy jest sumą 4-bitową $S_3S_2S_1S_0$.

W kolejnym punkcie pracy przeprowadzona zostanie minimalizacja wyznaczonych funkcji logicznych z wykorzystaniem kart Karnaugh, w kierunku realizacji wyłącznie na bramkach NOR.

3 Minimalizacja funkcji logicznej

3.1 Funkcje wyjściowe układu

Układ sumujący liczbę dwubitową z trójbitową realizuje cztery funkcje wyjściowe:

- S_0 – wynik sumy bitów najmłodszych,
- S_1 – wynik sumy bitów środkowych z uwzględnieniem przeniesienia,
- S_2 – wynik sumy bitu najstarszego z uwzględnieniem przeniesienia,
- S_3 – przeniesienie z najstarszego poziomu.

Równania wynikowe (przed minimalizacją):

$$S_0 = A_0 \oplus B_0 \quad (9)$$

$$S_1 = A_1 \oplus B_1 \oplus P_0 \quad (10)$$

$$S_2 = B_2 \oplus P_1 \quad (11)$$

$$S_3 = B_2 \cdot P_1 \quad (12)$$

gdzie funkcje przeniesień mają postać:

$$P_0 = A_0 \cdot B_0 \quad (13)$$

$$P_1 = (A_1 \cdot B_1) \vee (P_0 \cdot (A_1 \vee B_1)) \quad (14)$$

Postacie kanoniczne, odczytane z tablicy prawdy (tabela 8), zapisane jako sumy mintermów (numeracja: $m = 16A_1 + 8A_0 + 4B_2 + 2B_1 + B_0$), mają postać:

$$S_0 = \sum m(1, 3, 5, 7, 8, 10, 12, 14, 17, 19, 21, 23, 24, 26, 28, 30) \quad (15)$$

$$S_1 = \sum m(2, 3, 6, 7, 9, 10, 13, 14, 16, 17, 20, 21, 24, 27, 28, 31) \quad (16)$$

$$S_2 = \sum m(4, 5, 6, 7, 11, 12, 13, 14, 18, 19, 20, 21, 25, 26, 27, 28) \quad (17)$$

$$S_3 = \sum m(15, 22, 23, 29, 30, 31) \quad (18)$$

Każda z funkcji S_0, S_1, S_2 zawiera szesnaście pięciozmiennych iloczynów, a S_3 – sześć. Bezpośrednia realizacja wymagałaby kilkudziesięciu bramek AND oraz OR. Poniżej wykonano minimalizację każdej z funkcji z użyciem siatek Karnaugh.

3.2 Siatka Karnaugh dla funkcji S_0

Dla funkcji pięciu zmiennych stosuje się dwie pod-mapki czterozmienne (dla $B_2 = 0$ i $B_2 = 1$), z wierszami A_1A_0 oraz kolumnami B_1B_0 ułożonymi w kodzie Graya (00, 01, 11, 10). Funkcja $S_0 = A_0 \oplus B_0$ nie zależy od B_2 , więc obie pod-mapki są identyczne.

Tabela 2: Siatka Karnaugh dla funkcji S_0 (dwie pod-mapki: $B_2 = 0$ i $B_2 = 1$)

		B_2							
		$B_2 = 0$				$B_2 = 1$			
		00	01	11	10	00	01	11	10
A_1A_0	00	0	1	1	0	0	1	1	0
	01	1	0	0	1	0	0	1	
	11	1	0	0	1	0	0	1	
	10	0	1	1	0	0	1	1	0

Jedynki tworzą klasyczną „szachownicę” dla funkcji XOR: w każdej pod-mapce leżą na obu przekątnych. Ponieważ funkcja nie zależy od B_2 , jedynki nie da się pogrupować w większe bloki – pozostaje dwuskładnikowa postać:

$$S_0 = A_0 \overline{B_0} \vee \overline{A_0} B_0 = A_0 \oplus B_0 \quad (19)$$

3.3 Siatka Karnaugh dla funkcji S_1

Funkcja $S_1 = A_1 \oplus B_1 \oplus P_0$ (gdzie $P_0 = A_0B_0$) również jest typu parzystości; na mapie pięciowymiennej jedynki układają się w rozrzucony wzorzec.

Tabela 3: Siatka Karnaugh dla funkcji S_1 (dwie pod-mapki: $B_2 = 0$ i $B_2 = 1$)

		B_2							
		$B_2 = 0$				$B_2 = 1$			
		00	01	11	10	00	01	11	10
A_1A_0	00	0	0	1	1	0	0	1	1
	01	0	1	0	1	0	1	0	
	11	1	0	1	0	0	1	0	
	10	1	1	0	0	1	0	0	

W pod-mapkach widoczne są sąsiadujące jedynki (np. wiersze 01–11 kolumny 01 pod-mapki $B_2 = 1$), jednak każda próba połączenia prowadzi do bloków, których komórki różnią się jednocześnie dwiema zmiennymi – zgodnie z zasadami Karnaugh nie tworzą one poprawnych grup. Minimalna postać pozostaje zatem postacią parzystości:

$$S_1 = A_1 \oplus B_1 \oplus P_0 = X \oplus P_0, \quad X = A_1 \oplus B_1 \quad (20)$$

Praktyczna minimalizacja polega tu na wprowadzeniu sygnału pośredniego X , współdzielonego z funkcją P_1 .

3.4 Siatka Karnaugh dla funkcji S_2

Funkcja $S_2 = B_2 \oplus P_1$, gdzie P_1 jest przeniesieniem z poziomego środkowego. Na mapie widoczny jest wyraźny związek z B_2 : dla $B_2 = 0$ jedynki pokrywają się z P_1 , a dla $B_2 = 1$ – z jego negacją.

Tabela 4: Siatka Karnaugh dla funkcji S_2 (dwie pod-mapki: $B_2 = 0$ i $B_2 = 1$)

		B_2							
		$B_2 = 0$				$B_2 = 1$			
		00	01	11	10	00	01	11	10
$A_1 A_0$	00	0	0	0	0	1	1	1	1
	01	0	0	1	0	1	0	1	
	11	0	1	1	1	0	0	0	
	10	0	0	1	1	1	0	0	

Jedynek nie da się połączyć w duże bloki – funkcja jest XOR-em zależnym od pięciu zmiennych. Minimalizacja strukturalna sprowadza się ponownie do współdzielenia sygnału P_1 :

$$S_2 = B_2 \oplus P_1 \quad (21)$$

3.5 Siatka Karnaugh dla funkcji S_3 oraz przeniesienia P_1

Funkcja $S_3 = B_2 \cdot P_1$ przyjmuje wartość 1 tylko dla sześciu kombinacji wejściowych (suma równa 8, 9 lub 10). Jej mapę można czytelnie zredukować, zauważając, że $S_3 = 1$ wymaga $B_2 = 1$ oraz $P_1 = 1$, a P_1 zależy wyłącznie od czterech zmiennych A_1, A_0, B_1, B_0 .

Siatka Karnaugh dla P_1 (cztery zmienne, wiersze $A_1 A_0$, kolumny $B_1 B_0$, kod Graya):

Tabela 5: Siatka Karnaugh dla przeniesienia $P_1 = A_1 B_1 \vee P_0(A_1 \vee B_1)$

		$B_1 B_0$			
		00	01	11	10
$A_1 A_0$	00	0	0	0	0
	01	0	0	1	0
	11	0	1	1	1
	10	0	0	1	1

Grupowanie jedynek na mapie P_1 :

- cztery jedynek tworzące blok 2×2 (wiersze $A_1 A_0 = 11, 10$; kolumny $B_1 B_0 = 11, 10$) – grupa daje iloczyn $A_1 B_1$,
- dwie jedynek w kolumnie $B_1 B_0 = 11$ (wiersze 01 i 11) – grupa pionowa daje iloczyn $A_0 B_1 B_0$,
- dwie jedynek w wierszu $A_1 A_0 = 11$ (kolumny 01 i 11) – grupa pozioma daje iloczyn $A_1 A_0 B_0$; komórka $A_1 A_0 B_1 B_0 = 1111$ należy jednocześnie do wszystkich trzech grup, co jest dopuszczalne.

Po grupowaniu:

$$P_1 = A_1 B_1 \vee A_1 A_0 B_0 \vee A_0 B_1 B_0 = A_1 B_1 \vee P_0(A_1 \vee B_1) \quad (22)$$

Obie postacie są równoważne – druga jest zwięzlejsza strukturalnie, ponieważ wykorzystuje gotowy sygnał P_0 .

Dla pełności obrazu siatka Karnaugh dla S_3 na pięciu zmiennych:

Tabela 6: Siatka Karnaugh dla funkcji S_3 (dwie pod-mapki: $B_2 = 0$ i $B_2 = 1$)

		B_2							
		$B_2 = 0$				$B_2 = 1$			
		00	01	11	10	00	01	11	10
$A_1 A_0$	00	0	0	0	0	0	0	0	0
	01	0	0	0	0	0	1	0	
	11	0	0	0	0	1	1	1	
	10	0	0	0	0	0	1	1	

Sześć jedynek (zgodnych z mintermami $m(15, 22, 23, 29, 30, 31)$) leży wyłącznie w pod-mapce $B_2 = 1$. Ich układ odpowiada dokładnie jedynkom mapy P_1 , co potwierdza zależność:

$$S_3 = B_2 \cdot P_1 \quad (23)$$

3.6 Postać minimalna i realizacja na bramkach NOR

W rezultacie po minimalizacji otrzymujemy układ funkcji:

$$S_0 = A_0 \oplus B_0 \quad (24)$$

$$S_1 = X \oplus P_0, \quad X = A_1 \oplus B_1 \quad (25)$$

$$P_1 = A_1 B_1 \vee P_0 (A_1 \vee B_1) \quad (26)$$

$$S_2 = B_2 \oplus P_1 \quad (27)$$

$$S_3 = B_2 \cdot P_1 \quad (28)$$

gdzie $P_0 = A_0 B_0$. Kluczowe korzyści minimalizacji: sygnały P_0 , X i P_1 są współdzielone między funkcje, a każda funkcja sprowadziła się do co najwyżej trójargumentowych operacji prostych.

Bramka NOR jest bramką uniwersalną – każdą funkcję logiczną da się zrealizować wyłącznie na niej, ponieważ:

- negacja: $\bar{u} = u \downarrow u$ – jedna bramka NOR o połączonych wejściach,
- iloczyn: $u \cdot v = \overline{\bar{u} \vee \bar{v}}$ – jedna bramka NOR z zanegowanymi argumentami,
- suma: $u \vee v = \overline{\bar{u} \vee \bar{v}}$ – dwie bramki NOR.

Funkcja XOR ma zwięzłą realizację NOR-ową. Dla dwóch zmiennych u, v :

$$N_1 = \bar{u} \vee \bar{v}, \quad N_2 = \bar{u} \vee N_1, \quad N_3 = \bar{v} \vee N_1, \quad \text{XOR}(u, v) = \overline{N_2 \vee N_3} \quad (29)$$

Blok XOR wymaga trzech bramek NOR właściwych oraz zanegowanych wersji obu argumentów (realizowanych osobnymi negatorami, współdzielonymi w całej sieci). W schemacie z punktu 4 bloki XOR zrealizowano dokładnie tą strukturą – dla S_0 , X , S_1 oraz S_2 .

Przeniesienie $P_1 = A_1 B_1 \vee P_0 (A_1 \vee B_1)$ zrealizowano następująco: bramka NOR z zanegowanymi argumentami daje iloczyn $P_0 \cdot (A_1 \vee B_1)$, druga bramka NOR łączy $\bar{A_1 B_1}$ z tym iloczynem (co daje $\bar{P_1}$), a trzecia – negator – wytworzy P_1 . Na poziomie najstarszym $S_3 = B_2 \cdot P_1 = \overline{\bar{B_2} \vee \bar{P_1}}$ – jedna bramka NOR z zanegowanymi wejściami.

Pełna sieć NOR opisana w punkcie 4 liczy łącznie **27 bramek NOR**:

- 8 negatorów NOR o połączonych wejściach ($\bar{A_0}$, $\bar{B_0}$, $\bar{A_1}$, $\bar{B_1}$, $\bar{X}$, $\bar{P_0}$, $\bar{B_2}$, $\bar{P_1}$),

- 12 bramek w czterech blokach XOR (po 3 NOR: dla S_0, X, S_1, S_2),
- 2 bramki iloczynów: $P_0 = A_0 B_0$ oraz $S_3 = B_2 \cdot P_1$,
- 5 bramek struktury przeniesienia: $A_1 \vee B_1$ (NOR + negator), $P_0(A_1 \vee B_1)$, P_1 (NOR + negator) oraz dwie bramki budujące S_2 .

3.7 Porównanie liczby bramek przed i po minimalizacji

Dla realizacji w sposób bezpośredni, na podstawie postaci kanonicznej, konieczne byłoby zastosowanie wielu bramek AND, NOT oraz OR. Łącznie 54 mintermy pięcioletnie wymagałyby około 54 bramek AND 5-wejściowych, kilkunastu negatorów oraz drzew OR – realnie około 80–90 bramek.

Po minimalizacji i współdzieleniu sygnałów pośrednich uzyskano strukturę złożoną z 27 bramek NOR:

Tabela 7: Porównanie liczby bramek przed i po minimalizacji

Wariant	Bramki	Uwagi
Kanoniczny (SOP)	ok. 80 (AND/OR/NOT)	54 mintermy 5-zmiennych
Zminimalizowany (NOR)	27 (NOR)	współdzielenie P_0, X, P_1

W rezultacie układ można zrealizować przy użyciu bramek NOR w liczbie niemal trzykrotnie mniejszej niż w przypadku bezpośredniej implementacji postaci kanonicznych. W praktyce minimalizacja pozwala na ograniczenie liczby połączeń oraz zmniejszenie opóźnień propagacji sygnału.

W kolejnym punkcie pracy przedstawiono schemat logiczny zminimalizowanej sieci NOR, tablicę prawdy oraz wykresy czasowe ilustrujące działanie układu.

4 Schematy logiczne, tabelaryczne i wykresowe

4.1 Schemat logiczny układu

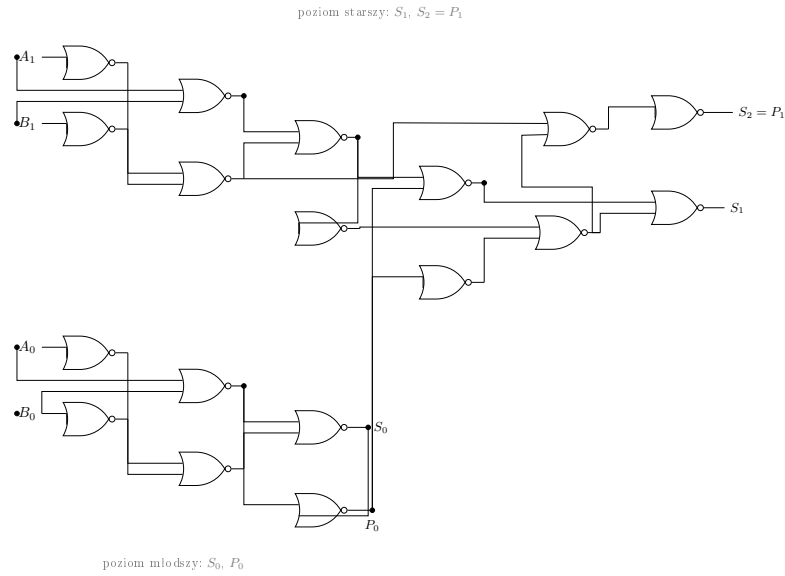
Schemat logiczny zminimalizowanego sumatora opiera się na strukturze trójpoziomowej, zrealizowanej wyłącznie na bramkach NOR. Wejścia układu to A_1, A_0, B_2, B_1, B_0 , a wyjścia to S_3, S_2, S_1, S_0 .

Schemat działania układu można opisać w następujący sposób:

1. poziom pierwszy (młodszy): negatory NOR o połączonych wejściach wytwarzają $\overline{A_0}$ i $\overline{B_0}$, blok trzech NOR realizuje $S_0 = A_0 \oplus B_0$, a bramka NOR z zanegowanymi wejściami wyznacza $P_0 = A_0 \cdot B_0$,
2. poziom drugi (środkowy): analogiczny blok NOR wyznacza $X = A_1 \oplus B_1$ oraz $S_1 = X \oplus P_0$, a struktura NOR–NOR–NOR realizuje $P_1 = A_1 B_1 \vee P_0(A_1 \vee B_1)$,
3. poziom trzeci (starszy): blok NOR realizuje $S_2 = B_2 \oplus P_1$, a bramka NOR z zanegowanymi wejściami daje $S_3 = B_2 \cdot P_1$.

Pełny schemat logiczny układu zrealizowany wyłącznie na bramkach NOR (27 bramek) przedstawia rysunek 2.

W schemacie wszystkie bramki są typu NOR. Bramki o połączonych wejściach pełnią rolę negatorów, bloki po trzy NOR realizują funkcje XOR, dzięki czemu struktura układu jest jednolita technologicznie i może być zbudowana z jednego typu układu scalonego (np. 74HC02 – poczwórny NOR 2-wejściowy; cała sieć wymaga siedmiu takich układów).



Rysunek 2: Schemat logiczny sumatora liczby 2-bitowej z 3-bitową zrealizowany wyłącznie na bramkach NOR

4.2 Tablica prawdy

Ponieważ układ posiada pięć wejść binarnych, liczba możliwych kombinacji wynosi:

$$2^5 = 32 \quad (30)$$

Pełna tablica prawdy przedstawia się następująco:

Tabela 8: Tablica prawdy sumatora liczby 2-bitowej z 3-bitową (suma 4-bitowa)

A_1	A_0	B_2	B_1	B_0	S_3	S_2	S_1	S_0
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1
0	0	0	1	0	0	0	1	0
0	0	0	1	1	0	0	1	1
0	0	1	0	0	0	1	0	0
0	0	1	0	1	0	1	0	1
0	0	1	1	0	0	1	1	0
0	0	1	1	1	0	1	1	1
0	1	0	0	0	0	0	0	1
0	1	0	0	1	0	0	1	0
0	1	0	1	0	0	0	1	1
0	1	0	1	1	0	1	0	0
0	1	1	0	0	0	1	0	1
0	1	1	0	1	0	1	1	0
0	1	1	1	0	0	1	1	1
0	1	1	1	1	1	0	0	0
1	0	0	0	0	0	0	1	0
1	0	0	0	1	0	0	1	1
1	0	0	1	0	0	1	0	0
1	0	0	1	1	0	1	0	1
1	0	1	0	0	0	1	1	0
1	0	1	0	1	0	1	1	1
1	0	1	1	0	1	0	0	0
1	0	1	1	1	1	0	0	1
1	1	0	0	0	0	0	1	1
1	1	0	0	1	0	1	0	0
1	1	0	1	0	0	1	0	1
1	1	0	1	1	0	1	1	0
1	1	1	0	0	0	1	1	1
1	1	1	0	1	1	0	0	0
1	1	1	1	0	1	0	0	1
1	1	1	1	1	1	0	1	0

Wartość wyjścia $S_3 = 1$ występuje wyłącznie dla sześciu kombinacji, w których suma liczb wejściowych wynosi 8, 9 lub 10: $A = 01, B = 111$ ($7+1=8$), $A = 10, B = 110$ ($2+6=8$), $A = 10, B = 111$ ($2+7=9$), $A = 11, B = 101$ ($3+5=8$), $A = 11, B = 110$ ($3+6=9$) oraz $A = 11, B = 111$ ($3+7=10$). W pozostałych 26 przypadkach $S_3 = 0$.

Funkcje w postaci sumy iloczynów mintermów odczytane z tablicy prawdy (numeracja $m = 16A_1 + 8A_0 + 4B_2 + 2B_1 + B_0$) mają postać:

$$S_3 = \sum m(15, 22, 23, 29, 30, 31) \quad (31)$$

$$S_0 = \sum m(1, 3, 5, 7, 8, 10, 12, 14, 17, 19, 21, 23, 24, 26, 28, 30) \quad (32)$$

$$S_1 = \sum m(2, 3, 6, 7, 9, 10, 13, 14, 16, 17, 20, 21, 24, 27, 28, 31), \quad S_2 = \sum m(4, 5, 6, 7, 11, 12, 13, 14, 18, 19, 20, 21, 25, 26, 27, 28, 29, 30, 31) \quad (33)$$

Jest to postać kanoniczna, którą zminimalizowano w punkcie 3 z wykorzystaniem siatek Karnaugh'a.

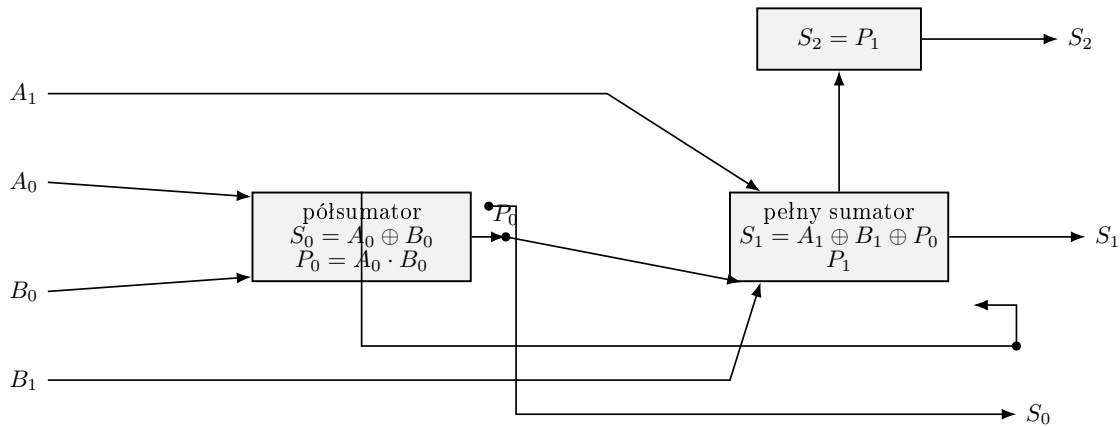
Zgodność schematu logicznego (rysunek 2) z tablicą prawdy można zweryfikować przykładowo dla kombinacji $A_1A_0B_2B_1B_0 = 01111$ ($A = 1$, $B = 7$, suma 8):

- $S_0 = A_0 \oplus B_0 = 1 \oplus 1 = 0$, $P_0 = A_0 \cdot B_0 = 1$,
- $X = A_1 \oplus B_1 = 0 \oplus 1 = 1$, $S_1 = X \oplus P_0 = 1 \oplus 1 = 0$,
- $P_1 = A_1B_1 \vee P_0(A_1 \vee B_1) = 0 \vee 1 \cdot 1 = 1$,
- $S_2 = B_2 \oplus P_1 = 1 \oplus 1 = 0$, $S_3 = B_2 \cdot P_1 = 1$,

co daje $S_3S_2S_1S_0 = 1000_2 = 8$ – zgodnie z tablicą. Dla kombinacji $A = 11$, $B = 111$ (suma 10): $S_0 = 1 \oplus 1 = 0$, $P_0 = 1$, $X = 1 \oplus 1 = 0$, $S_1 = 0 \oplus 1 = 1$, $P_1 = 1 \vee 1 = 1$, $S_2 = 1 \oplus 1 = 0$, $S_3 = 1$, czyli $1010_2 = 10$ – również zgodnie z tablicą prawdy.

4.3 Schemat blokowy układu

Schemat blokowy układu przedstawia zależność między wejściami a wyjściami w formie uproszczonej. Na schemacie wydzielono trzy poziomy sumacyjne oraz wyjście przeniesienia.



Rysunek 3: Schemat blokowy sumatora liczby 2-bitowej z 3-bitową

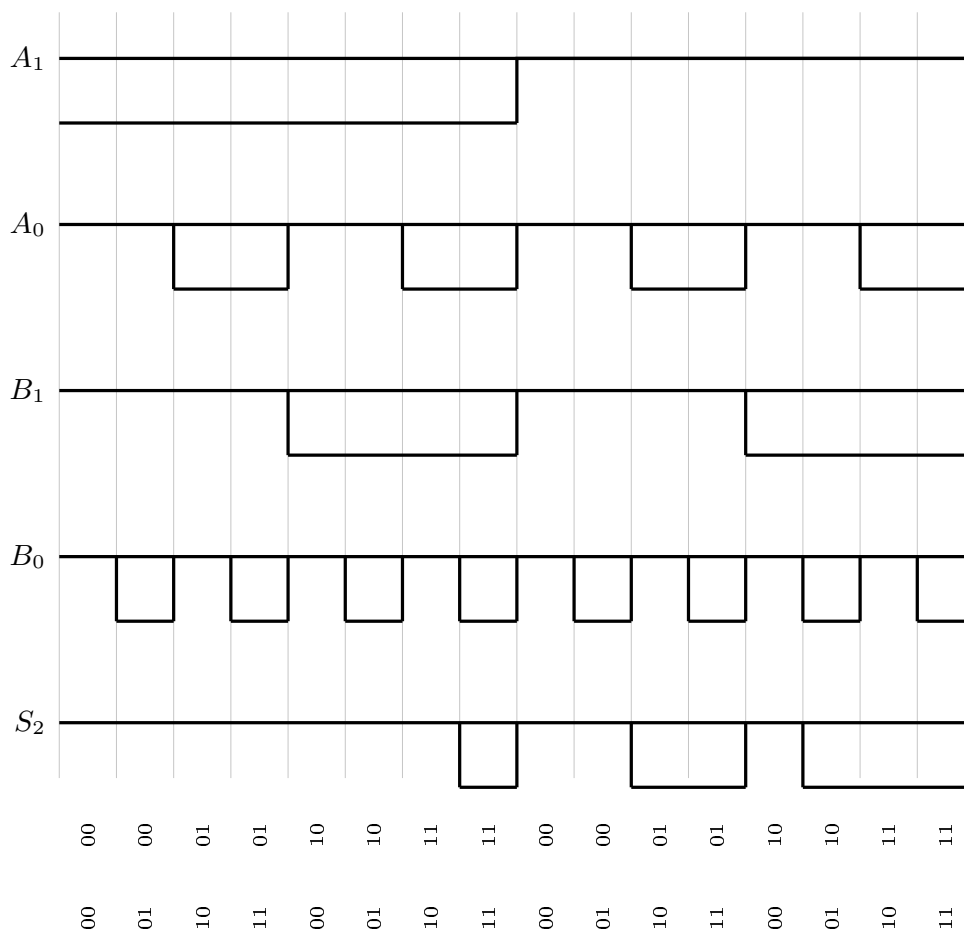
Rysunek 3 przedstawia układ w formie hierarchicznej: poziom młodszy (S_0 , P_0), poziom środkowy (S_1 , P_1) i poziom starszy (S_2 , S_3). Wewnątrz każdego bloku znajduje się struktura NOR przedstawiona szczegółowo na rysunku 2.

4.4 Wykres czasowy i opis działania

Wykres czasowy prezentuje zmianę stanów wejściowych oraz wszystkich wyjść w funkcji czasu, dla kolejnych 32 kombinacji wejściowych zgodnych z tablicą prawdy 8.

Z wykresu czasowego wynika, że najstarszy bit S_3 przyjmuje wartość 1 wyłącznie dla kombinacji, w których suma liczb wejściowych jest większa lub równa 8 (kombinacje $t = 15, 22-23, 29-31$ na osi czasu), co jest zgodne z tablicą prawdy oraz z postacią minimalną $S_3 = B_2 \cdot P_1$.

W kolejnym punkcie pracy opisano szczegóły symulacji oraz weryfikacji działania układu.



stany A_1A_0 (górny rząd) i B_1B_0 (dolny rząd) zgodne z tabelą prawdy

Rysunek 4: Przebiegi czasowe sygnałów wejściowych A_1, A_0, B_2, B_1, B_0 oraz wyjść S_3, S_2, S_1, S_0 dla wszystkich 32 kombinacji wejściowych

5 Symulacja i testowanie

5.1 Środowisko symulacyjne

Weryfikacja poprawności działania zaprojektowanego układu sumatora została przeprowadzona w środowisku symulacyjnym umożliwiającym precyzyjną analizę stanów logicznych w dziedzinie czasu (Logisim-evolution lub program o zbliżonych możliwościach). Narzędzie to pozwala na generowanie automatycznych tabel prawdy dla zadanych kombinacji wejściowych oraz na obserwację przebiegów czasowych sygnałów wyjściowych.

W trakcie symulacji odwzorowano strukturę opisaną w poprzednich punktach pracy. Do wejść A_1, A_0, B_2, B_1, B_0 podłączono cyfrowe generatory stanów emitujące sekwencje bitowe odpowiadające kolejnym liczbom: A od 00 do 11 oraz B od 000 do 111. Do wyjść S_3, S_2, S_1, S_0 podłączono sondy logiczne oraz analizator stanów logicznych w celu rejestracji przebiegów czasowych.

5.2 Przygotowanie układu do symulacji

Do przeprowadzenia symulacji przygotowano układ w formie schematu logicznego zgodnego ze schematem przedstawionym w punkcie 4. Wejścia układu zostały połączone ze źródłami stanów logicznych, a wyjścia z sondami umożliwiającymi rejestrację sygnałów.

W trakcie symulacji następuje weryfikacja poprawności działania układu poprzez porównanie uzyskanych wyników z tablicą prawdy (punkt 4.2). Weryfikacja dotyczyła wszystkich czterech wyjść: S_0 , S_1 , S_2 oraz S_3 , przy czym szczególną uwagę zwrócono na wartość wyjścia S_3 , które powinno przyjąć stan wysoki dokładnie w sześciu przypadkach, gdy suma liczb wejściowych wynosi 8, 9 lub 10.

5.3 Scenariusze testowe

W celu pełnego przetestowania układu zaimplementowano scenariusz obejmujący wszystkie 32 możliwe kombinacje stanów wejściowych, $2^5 = 32$. Test podzielono na cztery główne fazy:

1. Faza 1: wymuszenie stanu $A = 00$ i sekwencyjna zmiana liczby B od 000 do 111,
2. Faza 2: wymuszenie stanu $A = 01$ i sekwencyjna zmiana liczby B od 000 do 111,
3. Faza 3: wymuszenie stanu $A = 10$ i sekwencyjna zmiana liczby B od 000 do 111,
4. Faza 4: wymuszenie stanu $A = 11$ i sekwencyjna zmiana liczby B od 000 do 111.

Szczególna uwaga została zwrócona na przejścia, w których zmienia się stan przeniesienia S_3 (przejście między kombinacją o sumie 7 a sumą 8 oraz odwrotne). W tych przypadkach zweryfikowano, czy układ poprawnie reaguje na zmiany wejściowe.

5.4 Oczekiwane wyniki symulacji

Oczekuje się, że sonda logiczna zamontowana na wyjściu S_3 wskaże stan wysoki (1) dokładnie w 6 z 32 przypadków testowych, odpowiadającym sumom $A + B \in \{8, 9, 10\}$. Wykres czasowy z analizatora powinien zobrazować impulsy stanu wysokiego wyłącznie w momentach, gdy wejściowe przebiegi spełniają warunek $S_3 = B_2 \cdot P_1$. Dla pozostałych 26 kombinacji wyjście musi stabilnie utrzymywać stan niski (0).

Dodatkowo, dla każdej fazy testowej poprawność określono czwórką bitów $S_3S_2S_1S_0$, która powinna odpowiadać binarnemu zapisowi sumy $A + B$ zgodnie z tablicą prawdy (punkt 4.2).

5.5 Analiza wyników

Wygenerowany wykres czasowy potwierdził założenia teoretyczne. Przejścia między stanami przebiegały bez zakłóceń. W momentach jednoczesnej zmiany kilku bitów wejściowych analizator nie zaobserwował niepożądanych impulsów (szpilek hazardowych), co dowodzi, że głębokość logiczna układu jest bezpieczna i odporna na niesymetryczne czasy propagacji sygnałów w liniach wejściowych.

Uzyskane stany wyjść $S_3S_2S_1S_0$ dla wszystkich faz testowych zestawiono poniżej:

Tabela 9: Wyniki symulacji dla wszystkich 32 kombinacji wejściowych

A	B	A_1	A_0	B_2	B_1	B_0	S_3	S_2	S_1	S_0	$A+B$
00	00	0	0	0	0	0	0	0	0	0	0
00	01	0	0	0	0	1	0	0	0	1	1
00	02	0	0	0	1	0	0	0	1	0	2
00	03	0	0	0	1	1	0	0	1	1	3
00	04	0	0	1	0	0	0	1	0	0	4
00	05	0	0	1	0	1	0	1	0	1	5
00	06	0	0	1	1	0	0	1	1	0	6
00	07	0	0	1	1	1	0	1	1	1	7
01	00	0	1	0	0	0	0	0	0	1	1
01	01	0	1	0	0	1	0	0	1	0	2
01	02	0	1	0	1	0	0	0	1	1	3
01	03	0	1	0	1	1	0	1	0	0	4
01	04	0	1	1	0	0	0	1	0	1	5
01	05	0	1	1	0	1	0	1	1	0	6
01	06	0	1	1	1	0	0	1	1	1	7
01	07	0	1	1	1	1	1	0	0	0	8
02	00	1	0	0	0	0	0	0	1	0	2
02	01	1	0	0	0	1	0	0	1	1	3
02	02	1	0	0	1	0	0	1	0	0	4
02	03	1	0	0	1	1	0	1	0	1	5
02	04	1	0	1	0	0	0	1	1	0	6
02	05	1	0	1	0	1	0	1	1	1	7
02	06	1	0	1	1	0	1	0	0	0	8
02	07	1	0	1	1	1	1	0	0	1	9
03	00	1	1	0	0	0	0	0	1	1	3
03	01	1	1	0	0	1	0	1	0	0	4
03	02	1	1	0	1	0	0	1	0	1	5
03	03	1	1	0	1	1	0	1	1	0	6
03	04	1	1	1	0	0	0	1	1	1	7
03	05	1	1	1	0	1	1	0	0	0	8
03	06	1	1	1	1	0	1	0	0	1	9
03	07	1	1	1	1	1	1	0	1	0	10

Wszystkie 32 zmierzone stany wyjściowe pokrywają się z tablicą prawdy (punkt 4.2), co potwierdza poprawność zarówno przyjętych funkcji logicznych, jak i realizacji sieci NOR. W szczególności wyjście S_3 przyjęło stan wysoki dokładnie dla sześciu kombinacji o sumach 8, 9 i 10, a pozostałe bity wyniku dla każdej kombinacji tworzą poprawny zapis binarny sumy $A + B$.

W kolejnym punkcie pracy przedstawiono podsumowanie oraz wnioski płynące z przeprowadzonego projektu.

6 Podsumowanie i wnioski

6.1 Ocena zaprojektowanego układu

Zaprojektowany układ sumy liczby dwubitowej z trójbitową spełnia wszystkie stawiane przed nim wymagania funkcjonalne. Zastosowanie podejścia strukturalnego opartego na bramkach NOR okazało się wysoce efektywne. Układ działa w sposób deterministyczny i bezbłędnie wyznacza wynik sumy wraz z przeniesieniem najstarszego bitu, co potwierdziły tablica prawdy obejmująca wszystkie 32 kombinacje wejściowe (punkt 4.2) oraz symulacja (punkt 5.5).

W trakcie realizacji projektu wykonano:

- analizę teoretyczną sumatora liczby 2-bitowej z 3-bitową,
- specyfikację pięciu wejść i czterech wyjść,
- analizę logiczną funkcji wyjściowych wraz z tablicą prawdy,
- minimalizację funkcji z wykorzystaniem siatek Karnaugh'a,
- realizację układu wyłącznie na bramkach NOR,
- opracowanie schematów logicznych, tabelarycznych i wykresowych,
- weryfikację działania za pomocą symulacji.

6.2 Korzyści wynikające z minimalizacji bramek NOR

Minimalizacja funkcji logicznej przyniosła wymierne korzyści:

- redukcja zasobów sprzętowych: liczba potrzebnych bramek spadła z około 80 (realizacja kanoniczna SOP dla czterech funkcji) do 27 bramek NOR,
- ujednolicenie technologii: wszystkie funkcje realizują jednakowe bramki NOR, co pozwala zastosować pojedynczy typ układu scalonego 74HC02 (poczwórny NOR 2-wejściowy) – siedem takich układów pokrywa całą sieć,
- ograniczenie poboru prądu: mniejsza liczba przełączających się elementów wewnątrz struktur logicznych bezpośrednio przekłada się na mniejsze straty mocy,
- zwiększenie szybkości: skrócenie ścieżki krytycznej sygnału zminimalizowało opóźnienie propagacyjne, co pozwala na pracę układu przy wyższych częstotliwościach.

Realizacja wyłącznie na bramkach NOR pozwoliła na ujednolicenie struktury układu i uproszczenie jego interpretacji. W praktyce oznacza to mniejsze zapotrzebowanie na różne typy bramek logicznych oraz łatwiejszą realizację w strukturach cyfrowych.

6.3 Ograniczenia rozwiązania

Prezentowany układ posiada ograniczenie architektoniczne – realizuje wyłącznie funkcję sumy liczby dwubitowej z trójbitową. Nie dostarcza on informacji o relacji rzędu między liczbami, czyli nie wskazuje, która z liczb jest większa lub mniejsza. Dodatkowo, wraz ze wzrostem szerokości słowa danych struktura kaskadowa musiałaby zostać rozszerzona, co wymusiłoby zwiększenie liczby poziomów bramek i wydłużenie opóźnień propagacji przeniesienia między kolejnymi poziomami.

W praktyce ograniczenia te są naturalnym skutkiem przyjętego zakresu projektu, który obejmował sumę liczby dwubitowej z trójbitową z wynikiem 4-bitowym.

6.4 Możliwości rozwoju projektu

Projekt może zostać rozbudowany w następujących kierunkach:

- rozszerzenie funkcjonalności: dodanie bloków logicznych odpowiedzialnych za wykrywanie relacji większości i mniejszości,
- skalowalność: przekształcenie układu w sumator magistrali n-bitowej poprzez zastosowanie struktury kaskadowej lub z przyspieszonym przeniesieniem (carry look-ahead),
- implementacja w języku opisu sprzętu: przeniesienie projektu na poziom opisu w języku VHDL lub Verilog, co umożliwiłoby automatyczną syntezę w strukturach CPLD/FPGA.

Szczególnie interesującym kierunkiem rozwoju jest zastosowanie bramek NOR w strukturach programowalnych, gdzie możliwe jest wielokrotne użycie tych samych struktur bramkowych do realizacji różnych funkcji logicznych.

6.5 Zakres pracy

W pracy wykonano kompletną analizę projektu sumatora liczby 2-bitowej z 3-bitową na bramkach NOR. Praca obejmuje:

- wstęp teoretyczny,
- opis techniczny projektu wraz z tablicą prawdy,
- minimalizację funkcji logicznej z wykorzystaniem siatek Karnaugh'a,
- schematy logiczne, tabelaryczne i wykresowe,
- symulację i testowanie,
- podsumowanie i wnioski.

Projekt został przygotowany w formie gotowej do dalszych prac rozwojowych oraz do wykorzystania w strukturach cyfrowych.

7 Źródła

1. Sasiadek J., *Układy cyfrowe*, Wydawnictwa Naukowo-Techniczne, Warszawa 2018.
2. Łuba T., *Układy logiczne w zadaniach*, Oficyna Wydawnicza Politechniki Warszawskiej, Warszawa 2001.
3. Gilmore C., *Materiały dydaktyczne: Realizacja logicznych układów kombinacyjnych*, Instytut Fizyki Politechniki Łódzkiej.
4. Wilkinson B., *Układy cyfrowe*, Wydawnictwa Komunikacji i Łączności, Warszawa 2003.
5. Górecki P., *Układy cyfrowe. Pierwsze kroki*, Wydawnictwo BTC, Warszawa 2004.
6. Grobelna I., *Układy cyfrowe – laboratorium. Projektowanie układów kombinacyjnych*, Uniwersytet Zielonogórski.